

JavaScript-04b

Async/Await och Axios

30 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin

Introduktion till Angular JS

Nu har vi djupdykt i både React (kapitel 13–15) och Vue (kapitel 16). Men det finns en tredje jätte på marknaden som helt dominerar inom enterprise-världen – det vill säga hos storbanker, myndigheter och globala storföretag: Angular.

Medan React och Vue i grunden är flexibla bibliotek där du får plocka ihop dina egna verktyg för till exempel routing och API-anrop, är Angular ett komplett och strikt ramverk. Angular ger dig färdiga, inbyggda lösningar för precis allt. Det tvingar alla utvecklare i ett team att skriva sin kod på exakt samma sätt. Detta gör att enorma projekt kan växa i årtal utan att koden blir spretig eller rörig.

Kravet på TypeScript

Det första som skiljer Angular från React och Vue är att du inte skriver vanlig JavaScript. Angular är helt uppbyggt kring, och kräver, att du använder TypeScript.

TypeScript är en utbyggnad av JavaScript som lägger till statisk typning. Det innebär att du måste berätta för datorn exakt vilken datatyp alla variabler, funktionsargument och returvärden har (till exempel string, number eller boolean). Om du råkar skicka fel datatyp någonstans kommer koden att vägra att kompilera.

Om du har programmerat i Java eller C# tidigare kommer du att känna igen dig i Angular direkt!

Hur en Angular-komponent är uppbyggd

Precis som i React och Vue delar vi upp gränssnittet i små Lego-bitar (komponenter). I modern Angular bygger vi komponenter med hjälp av vanliga TypeScript-klasser som vi sätter en så kallad Decorator ovanpå.

En Decorator är en rad som startar med ett @-tecken (till exempel @Component). Den fungerar som en instruktion till Angular-motorn om hur klassen ska förvandlas till en fungerande webbkomponent under huven.

I modern Angular (version 17 och framåt) skriver vi så kallade Standalone Components. HTML-mallen, CSS-stylingen och koden bor ofta i samma fil, precis som i Vue:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-halsning', // Detta blir vår egna HTML-tagga: <app-halsning></app-halsning>
  standalone: true,          // Komponenten är självständig och behöver inga gamla moduler
  template: `
    <div class="box">
      <h1>Välkommen till Angular!</h1>
      <p>Inloggad användare: <b>{{ användare }}</b></p>

      <!-- Angular använder runda parenteser (click) för händelselyssnare -->
      <button (click)="bytAnvandare()">Byt användare</button>
    </div>
  `,
  styles: `.box { padding: 20px; background: #f1f3f5; border-radius: 6px; }`
})
export class HalsningComponent {
  // --- HÄR SKRIVER VI VÅR LOGIK (OBJEKTORIENTERAD TYPESCRIPT) ---

  // Vi måste tala om att variabeln är en sträng (string)
  användare: string = 'Acke';

  // Vi talar om att funktionen returnerar "void" (ingenting)
  bytAnvandare(): void {
    this.användare = 'Kalle';
  }
}
```

Bild 1 En Angular-komponent separerar metadata (i @Component) från själva affärslogiken (i klassen) på ett väldigt strukturerat sätt.

Modern reaktivitet under huven: Signals

I kapitel 15 såg vi hur React använder `useState` för att hantera sitt minne, och i kapitel 16 såg vi hur Vue använder `ref()`. I modern Angular hanterar vi numera reaktivitet med något som kallas för Signals (signaler).

En Signal är en reaktiv behållare som håller ett värde. Det fantastiska med signaler är att Angular vet exakt var på skärmen som signalen används. När du uppdaterar en signal behöver Angular inte rita om en hel komponent eller jämföra ett stort virtuellt HTML-träd – den går bara raka vägen till den specifika textraden i webbläsaren och uppdaterar den på ett par mikrosekunder!

Kodexempel med Signals (Klickräknare):

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-raknare',
  standalone: true,
  template: `
    <div class="counter">
      <!-- För att läsa en signal i mallen anropar vi den som en funktion () -->
      <h2>Du har klickat {{ antal() }} gånger</h2>
      <button (click)="plussaPa()">Klicka här</button>
    </div>
  `,
})
export class RaknareComponent {
  // Vi skapar en signal med ett startvärde på 0
  antal = signal<number>(0);

  plussaPa(): void {
    // För att uppdaterar en signal använder vi .update()
    this.antal.update(nuvarande => nuvarande + 1);
  }
}
```

Bild 2 Signals gör att Angular-appar drar extremt mycket mindre processorkraft och RAM-minne än traditionella ramverk vid stora datamängder.

Ramverk – vilket väljer man?

Nu när du har sett alla tre giganterna i kursen, vilket ramverk är egentligen bäst? Det beror helt och hållet på vilket projekt du ska bygga eller vilket företag du vill jobba på:

- React (Kapitel 13–15) Blandar HTML och JS i JSX-filer. Flexibelt men kräver att du installerar egna externa bibliotek. Perfekt för nystartade företag (startups), flexibla webbappar och om du vill ha tillgång till världens största jobbmarknad.
- Vue (Kapitel 16) Delar upp HTML, JS och CSS i SFC-filer (.vue). Har inbyggd "magi" som tvåvägs-databindning. Perfekt om du vill bygga projekt snabbt, föredrar ren HTML-struktur och vill ha ett ramverk som är lätt att lära sig.
- Angular (Kapitel 17) Använder TypeScript, klasser och strikt uppdelad enterprise-arkitektur. Allt ingår från start. Ett absolut måste för enorma system (t.ex. banker, försäkringsbolag eller tunga system i offentlig sektor) där ordning och reda är lag.